

HW 2

Sepehr Ilami

October 2025

Github Link: https://github.com/sepehrilami/phys7332_fa25/blob/main/assignments/assignment02.ipynb

Question 1

a) Redis

Data Structure:

Redis is an **in-memory key-value store**. Data is stored as pairs of keys and values, where values can be various data types such as strings, lists, sets, sorted sets, and hashes.

Best For:

Redis excels in **high-speed, real-time applications** requiring rapid read/write operations and low latency. Common use cases include:

- Caching frequently accessed data
- Session management
- Leaderboards and counters
- Real-time analytics

Bad Use Case:

Redis is **poorly suited for long-term data storage** or **complex querying** (e.g., relational joins or aggregations). Because it stores data in memory, large datasets can be expensive to maintain and are not persistent by default.

b) MongoDB

Data Structure:

MongoDB is a **document-oriented database** that stores data in **BSON (binary JSON)** format. Documents are grouped into collections, allowing for flexible, schema-less data storage.

Best For:

MongoDB is ideal for **semi-structured or evolving datasets**, where flexibility and scalability are important. Typical applications include:

- Content management systems
- E-commerce product catalogs
- Mobile or web app backends
- Real-time analytics dashboards

Bad Use Case:

MongoDB is **not well-suited for complex transactions or strong consistency requirements**

across multiple documents. Financial applications requiring atomic multi-table operations (like bank transfers) perform better in relational databases.

c) Cassandra

Data Structure:

Cassandra is a **wide-column store**, inspired by Google's Bigtable. Data is stored in rows and columns but grouped into **column families** instead of traditional tables. It is designed for distributed, horizontally scalable systems.

Best For:

Cassandra shines in **highly available, write-heavy environments** with massive, distributed datasets. Common use cases include:

- Time-series data (IoT sensors)
- Messaging platforms
- Recommendation systems
- Global-scale applications requiring fault tolerance

Bad Use Case:

Cassandra is **not ideal for ad-hoc queries or applications needing complex joins or aggregations**. Its query language is limited in analytical functionality, making it unsuitable for business intelligence systems or highly relational data.

d) Neo4j

Data Structure:

Neo4j is a **graph database** that stores data as **nodes, relationships, and properties**. It uses the Cypher query language to express graph patterns efficiently.

Best For:

Neo4j is built for **data that is highly interconnected**, where relationships are very important. It's actually ideal for network science and this course. Use cases include:

- Social networks
- Fraud detection
- Knowledge graphs
- Recommendation engines

Bad Use Case:

Neo4j is **poorly suited for applications centered on flat, tabular data** or those requiring complex aggregations across unrelated entities. For example, large-scale financial reporting or bulk numerical computations would perform better in a relational or analytical (OLAP) database.

Database	Data Model	Best For	Poor Fit
Redis	Key-value (in-memory)	Caching, real-time analytics	Long-term storage, complex queries
MongoDB	Document (BSON/JSON)	Flexible schemas, web apps	Multi-document transactions

Database	Data Model	Best For	Poor Fit
Cassandra	Wide-column	Distributed, high-write workloads	Complex queries, BI analytics
Neo4j	Graph (nodes + edges)	Relationship-heavy data	Flat or numerical datasets

Setup

```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
from collections import defaultdict

import scipy.sparse as sp
import time
import seaborn as sns
import pandas as pd
import os
import community as community_louvain
import warnings
warnings.filterwarnings("ignore")
import random
import scipy.io
from graph_tool.all import *
from sklearn.metrics import normalized_mutual_info_score
from matplotlib.ticker import ScalarFormatter, LogLocator
import matplotlib as mpl

mpl.rcParams['font.family'] = 'sans-serif'
mpl.rcParams['font.sans-serif'] = ['Arial', 'DejaVu Sans']
mpl.rcParams['font.size'] = 11
mpl.rcParams['axes.linewidth'] = 1.2
mpl.rcParams['xtick.major.width'] = 1.2
mpl.rcParams['ytick.major.width'] = 1.2
mpl.rcParams['xtick.minor.width'] = 0.8
mpl.rcParams['ytick.minor.width'] = 0.8
```

Question 2

a)

We will use an interaction network dataset. It's the email communication network at the University Rovira i Virgili in Tarragona in the south of Catalonia in Spain. Nodes are users and edges indicate that at least one email was sent.

Link: <https://networkrepository.com/email-univ.php>

citation:

Rossi, R. A., & Ahmed, N. K. (2015). The Network Data Repository with Interactive Graph Analytics and Visualization. In Proceedings of the AAAI Conference on Artificial Intelligence (Vol. 29, No. 1). <https://networkrepository.com>

Roger Guimera, Leon Danon, Albert Diaz-Guilera, Francesc Giralt, and Alex Arenas. 2003. Self-similar community structure in a network of human interactions. Physical review E 68, 6 (2003), 065103.

```
# load the data from data/ia-email-univ/ia-email-univ.mtx using
networkx
data = scipy.io.mmread('../data/ia-email-univ/ia-email-univ.mtx')

# Select a modularity maximization algorithm to detect communities in
this graph: Louvain
# Convert the sparse matrix to a NetworkX graph
G = nx.from_scipy_sparse_array(data)
G = G.to_undirected()
G.remove_edges_from(nx.selfloop_edges(G))
# if the graph is not connected, take the largest connected component
if not nx.is_connected(G):
    print("Graph is not connected. Taking the largest connected
component.")
    G = G.subgraph(max(nx.connected_components(G), key=len)).copy()
print(f"Graph has {G.number_of_nodes()} nodes and
{G.number_of_edges()} edges.")
# Ensure the graph is connected
if not nx.is_connected(G):
    raise ValueError("The graph is not connected.")

# Do community detection using Louvain method
start_time = time.time()
partition = community_louvain.best_partition(G, random_state=42)
end_time = time.time()
louvain_time = end_time - start_time
print(f"Louvain method took {louvain_time:.4f} seconds.")
# number of communities
num_communities = len(set(partition.values()))
print(f"Number of communities detected: {num_communities}")
```

```
Graph has 1133 nodes and 5451 edges.
Louvain method took 0.1150 seconds.
Number of communities detected: 10
```

Visualization

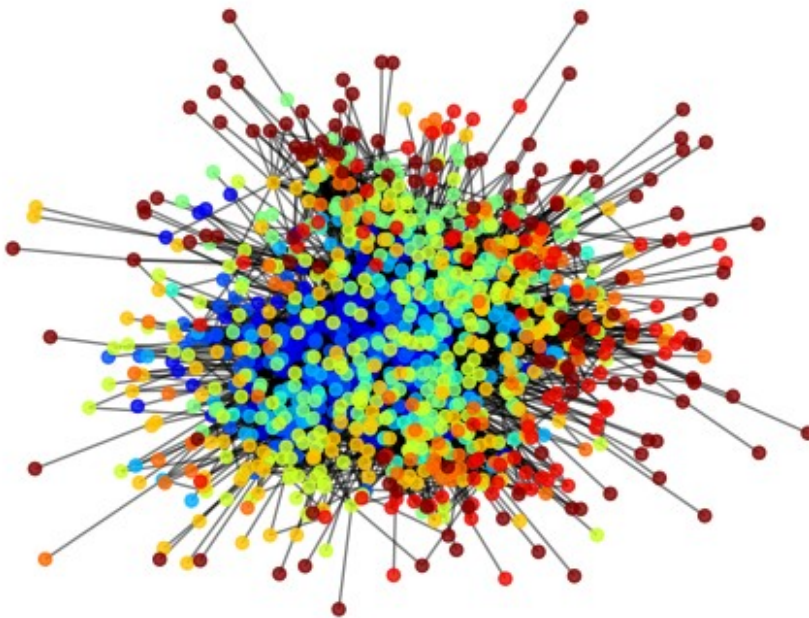
```
# Great visualization of the detected communities
# Set the color for each node based on its community
size = float(len(set(partition.values())))
pos = nx.spring_layout(G)
count = 0.
```

```

colors = []
for com in set(partition.values()):
    count = count + 1.
    list_nodes = [nodes for nodes in partition.keys() if
partition[nodes] == com]
    color = plt.cm.jet(count / size)
    for node in list_nodes:
        colors.append(color)
# Draw the graph
nx.draw_networkx_nodes(G, pos, node_size=20, node_color=colors,
alpha=0.8)
nx.draw_networkx_edges(G, pos, alpha=0.5)
plt.title("Louvain Community Detection")
plt.axis('off')
plt.show()

```

Louvain Community Detection



```

# Complicated visualization: Non-overlapping community layout
def plot_communities_separated(G, partition, seed=42,
                                cell_size=4.0, # spacing between
community centers
                                cluster_scale=0.9 # how much of the
cell to fill (0-1)
                                ):
    """
    Draw communities in a non-overlapping grid.

```

Each community gets a cell; nodes are spring-laid inside that cell and then shifted.

```
"""
# --- group nodes by community
comm2nodes = defaultdict(list)
for n, c in partition.items():
    comm2nodes[c].append(n)
communities = sorted(comm2nodes.keys())
C = len(communities)

# --- choose grid size (rows x cols)
cols = int(np.ceil(np.sqrt(C)))
rows = int(np.ceil(C / cols))

# --- community centers on a grid
centers = {}
idx = 0
for r in range(rows):
    for c in range(cols):
        if idx >= C: break
        cx = c * cell_size
        cy = -r * cell_size
        centers[communities[idx]] = (cx, cy)
        idx += 1

# --- colors (categorical)
base = plt.get_cmap("tab20").colors
palette = (list(base) * ((C // len(base)) + 1))[:C]
color_for = {comm: palette[i] for i, comm in
enumerate(communities)}

# --- compute inner layouts and compose global positions
rng = np.random.default_rng(seed)
pos = {}
for comm in communities:
    nodes = comm2nodes[comm]
    sub = G.subgraph(nodes)
    inner = nx.spring_layout(sub, seed=int(rng.integers(0,
1_000_000)))
    # normalize to a unit box so all communities fit similarly in
    # their cells
    arr = np.array(list(inner.values()))
    if len(arr) > 1:
        arr = (arr - arr.mean(0))
        denom = np.ptp(arr, axis=0) # NumPy 2.0 safe
        denom[denom == 0] = 1.0
        arr = arr / denom
    scale = (cell_size * 0.5) * cluster_scale
    cx, cy = centers[comm]
    for node, (x, y) in zip(nodes, arr):
```

```

        pos[node] = (cx + scale * x, cy + scale * y)

    # --- edge and node drawing
    plt.figure(figsize=(12, 10))
    nx.draw_networkx_edges(G, pos, alpha=0.08, width=0.4,
edge_color="#444444")

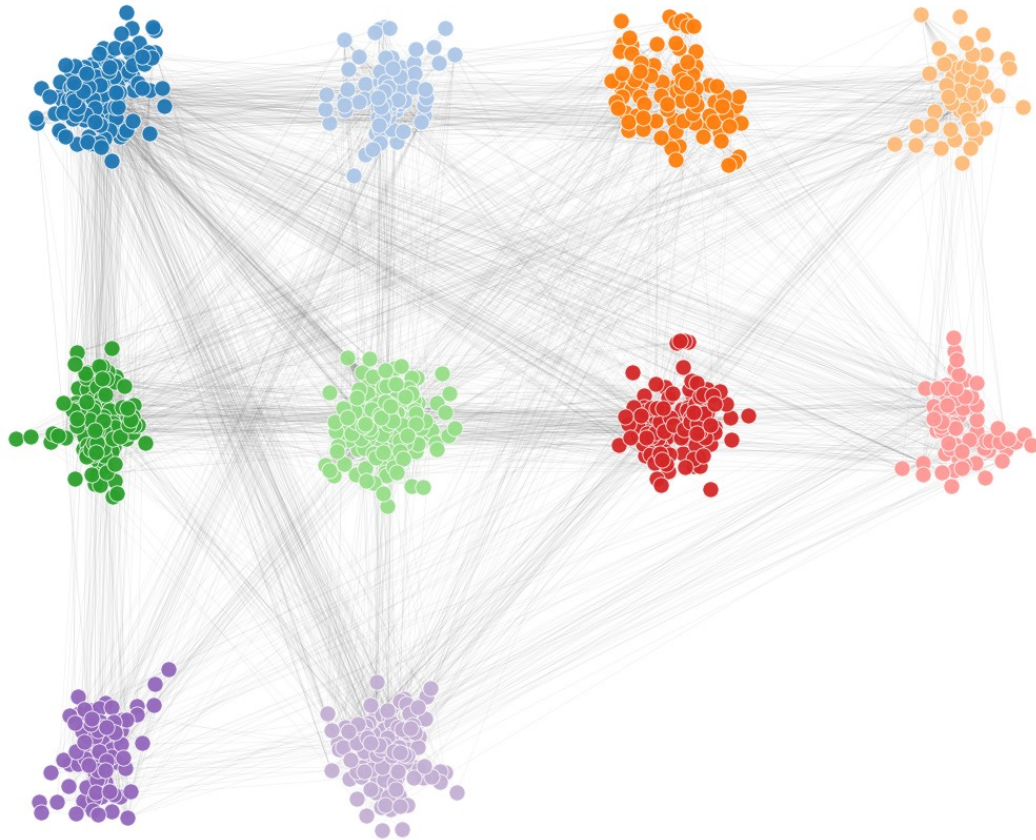
    for comm in communities:
        nodelist = comm2nodes[comm]
        nx.draw_networkx_nodes(
            G, pos,
            nodelist=nodelist,
            node_color=[color_for[comm]],
            node_size=120,
            edgecolors="white", linewidths=0.6, alpha=0.95
        )

    plt.axis("off")
    plt.title("Communities (non-overlapping grid layout)", pad=10)
    plt.tight_layout()
    plt.show()

plot_communities_separated(G, partition, seed=42, cell_size=4.0,
cluster_scale=0.9)

```

Communities (non-overlapping grid layout)



Modularity Calculation and Number of Communities

```
# Report the modularity of the partition your algorithm found
modularity = community_louvain.modularity(partition, G)
print(f"Modularity of the partition: {modularity:.4f}")
# Number of communities
num_communities = len(set(partition.values()))
print(f"Number of communities detected: {num_communities}")
```

```
Modularity of the partition: 0.5702
Number of communities detected: 10
```

Store data as a dictionary

```
# Store your partition as a dictionary, in the form of {node id:
community_id}
partition_dict = partition
# Save the partition to a text file, inside the other_stuff folder
if not os.path.exists('other_stuff'):
```



```

    os.makedirs('other_stuff')
with open('other_stuff/louvain_partition.txt', 'w') as f:
    for node, comm in partition_dict.items():
        f.write(f"{node}\t{comm}\n")
print("Louvain partition saved to other_stuff/louvain_partition.txt")
Louvain partition saved to other_stuff/louvain_partition.txt

```

b)

```

# --- COMMUNITY DETECTION WITH GRAPH-TOOL, THEN REUSE YOUR PLOTTER ---

import networkx as nx
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict

import graph_tool.all as gt  # <-- make sure graph-tool is installed

# 1) Helper: convert NetworkX -> graph-tool (keeps a name property for
mapping back)
def nx_to_gt(G):
    g = gt.Graph(directed=G.is_directed())
    v_name = g.new_vertex_property("object")
    g.vp["name"] = v_name

    nx2gt = {}
    for n in G.nodes():
        v = g.add_vertex()
        nx2gt[n] = v
        v_name[v] = n

    # (optional) pass weights if present
    has_w = any(("weight" in d) for _, _, d in G.edges(data=True))
    if has_w:
        e_w = g.new_edge_property("double")
        g.ep["weight"] = e_w

    for u, v, d in G.edges(data=True):
        e = g.add_edge(nx2gt[u], nx2gt[v])
        if has_w:
            g.ep["weight"][e] = float(d.get("weight", 1.0))

    return g, nx2gt

# 2) Community detection via SBM minimum description length
def gt_sbm_partition(G, seed=42, use_edge_weights=True):
    g, nx2gt = nx_to_gt(G)
    # If you want to consider weights, graph-tool handles them via
    "recs"/"rec_types"

```

```

# but a simple, robust starting point is to ignore weights:
state = gt.minimize_blockmodel_dl(g)
blocks = state.get_blocks()
partition = {n: int(blocks[nx2gt[n]]) for n in G.nodes()}
return partition, state, blocks

# 3) Your existing non-overlapping plotter (unchanged)
def plot_communities_separated(G, partition, seed=42,
                              cell_size=4.0,
                              cluster_scale=0.9):
    from collections import defaultdict
    comm2nodes = defaultdict(list)
    for n, c in partition.items():
        comm2nodes[c].append(n)
    communities = sorted(comm2nodes.keys())
    C = len(communities)

    cols = int(np.ceil(np.sqrt(C)))
    rows = int(np.ceil(C / cols))

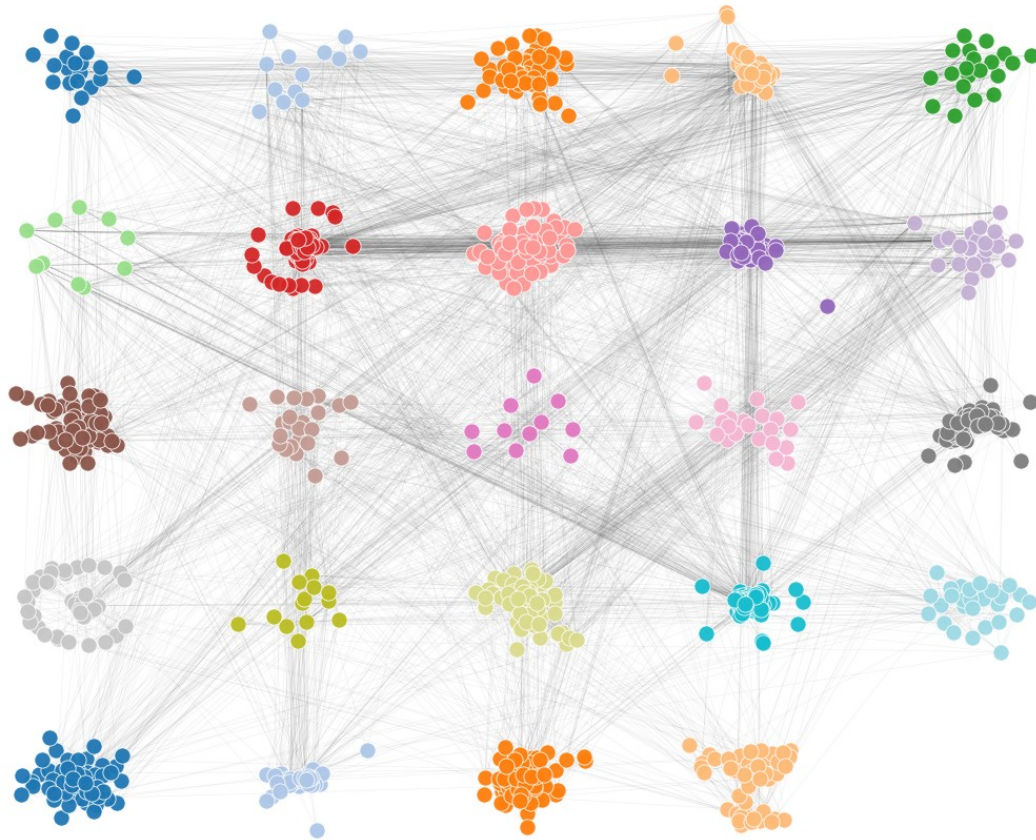
    centers, idx = {}, 0
    for r in range(rows):
        for c in range(cols):
            if idx >= C: break
            centers[communities[idx]] = (c * cell_size, -r *
cell_size)
            idx += 1

    base = plt.get_cmap("tab20").colors
    palette = (list(base) * ((C // len(base)) + 1))[:C]
    color_for = {comm: palette[i] for i, comm in
enumerate(communities)}

    rng = np.random.default_rng(seed)
    pos = {}
    for comm in communities:
        nodes = comm2nodes[comm]
        sub = G.subgraph(nodes)
        inner = nx.spring_layout(sub, seed=int(rng.integers(0,
1_000_000)))
        arr = np.array(list(inner.values()))
        if len(arr) > 1:
            arr = (arr - arr.mean(0))
            denom = np.ptp(arr, axis=0)
            denom[denom == 0] = 1.0
            arr = arr / denom
        scale = (cell_size * 0.5) * cluster_scale
        cx, cy = centers[comm]
        for node, (x, y) in zip(nodes, arr):
            pos[node] = (cx + scale * x, cy + scale * y)

```


Communities (graph-tool SBM) — non-overlapping layout



```
# Save the partition to a text file, inside the other_stuff folder
with open('other_stuff/graph_tool_partition.txt', 'w') as f:
    for node, comm in partition_gt.items():
        f.write(f"{node}\t{comm}\n")
print("Graph-tool SBM partition saved to
other_stuff/graph_tool_partition.txt")
```

```
# number of communities
num_communities_gt = len(set(partition_gt.values()))
print(f"Number of communities detected by graph-tool SBM:
{num_communities_gt}")
```

```
Graph-tool SBM partition saved to other_stuff/graph_tool_partition.txt
Number of communities detected by graph-tool SBM: 24
```

c)

```
G_random = nx.double_edge_swap(G.copy(), nswap=5*G.number_of_edges(),
max_tries=100*G.number_of_edges(), seed=42)
```

```

# Louvain on randomized graph
partition_louvain_random = community_louvain.best_partition(G_random,
random_state=42)
# Graph-tool SBM on randomized graph
partition_gt_random, state_random, blocks_random =
gt_sbm_partition(G_random)
# Save the partitions to text files
with open('other_stuff/louvain_partition_random.txt', 'w') as f:
    for node, comm in partition_louvain_random.items():
        f.write(f"{node}\t{comm}\n")
with open('other_stuff/graph_tool_partition_random.txt', 'w') as f:
    for node, comm in partition_gt_random.items():
        f.write(f"{node}\t{comm}\n")
# number of communities in randomized graph
num_communities_louvain_random =
len(set(partition_louvain_random.values()))
num_communities_gt_random = len(set(partition_gt_random.values()))
print(f"Number of communities detected by Louvain on randomized graph:
{num_communities_louvain_random}")
print(f"Number of communities detected by graph-tool SBM on randomized
graph: {num_communities_gt_random}")

```

```

Number of communities detected by Louvain on randomized graph: 12
Number of communities detected by graph-tool SBM on randomized graph:
1

```

d)

```

def _community_sizes(partition):
    _, counts = np.unique(list(partition.values()),
return_counts=True)
    return counts.astype(int)

def _ccdf_from_sizes(sizes):
    uniq, counts = np.unique(sizes, return_counts=True)
    tail_counts = np.cumsum(counts[::-1])[::-1]
    ccdf = tail_counts / tail_counts[0]
    return uniq, ccdf

def plot_community_size_distributions(partitions, titles,
save_path=None):
    """
    Create publication-ready community size distribution plots.

    Parameters:
    -----
    partitions : list of dict
        List of 4 partition dictionaries {node_id: community_id}
    titles : list of str
        List of 4 subplot titles
    """

```

```

    save_path : str, optional
        If provided, saves figure to this path (e.g., 'figure.pdf')
    """
    assert len(partitions) == 4 and len(titles) == 4, "Need exactly
four partitions and titles"

    # Color scheme: distinguish original (blues) vs randomized
    (oranges)
    colors = ['#2E5EAA', '#2E5EAA', '#D4661F', '#D4661F'] #
Colorblind-friendly
    linestyle = ['-', '--', '-', '--'] # Solid for Louvain, dashed
for SBM

    # Precompute for consistent axes
    all_sizes = [_community_sizes(p) for p in partitions]
    max_size = max(int(s.max()) for s in all_sizes)
    max_comms = max(len(s) for s in all_sizes)
    ymin = 0.8 / max_comms
    xmin, xmax = 0.8, max(2, max_size * 1.2)

    # Create figure with optimal size for two-column publication
    fig, axes = plt.subplots(2, 2, figsize=(9, 7), sharex=True,
sharey=True)
    axes = axes.ravel()

    for idx, (ax, sizes, title) in enumerate(zip(axes, all_sizes,
titles)):
        x, y = _ccdf_from_sizes(sizes)

        # Plot with publication styling
        ax.step(x, y, where="post", linewidth=2.5,
                color=colors[idx], linestyle=linestyle[idx],
                alpha=0.85, solid_capstyle='round')

        # Axis scaling
        ax.set_xscale("log")
        ax.set_yscale("log")
        ax.set_xlim(xmin, xmax)
        ax.set_ylim(ymin, 1.2)

        # Refined grid
        ax.grid(True, which="major", linestyle="-", linewidth=0.5,
alpha=0.3, color='gray')
        ax.grid(True, which="minor", linestyle=":", linewidth=0.3,
alpha=0.2, color='gray')

        # Title with bold font
        ax.set_title(title, pad=10, fontsize=11,
fontweight='semibold')

```

```

# Labels
if idx >= 2: # Bottom row
    ax.set_xlabel("Community size (nodes)", fontsize=11,
labelpad=5)
    if idx % 2 == 0: # Left column
        ax.set_ylabel(r"$P(S \geq s)$", fontsize=11, labelpad=5)

# Tick formatting
ax.xaxis.set_major_formatter(ScalarFormatter())
ax.yaxis.set_major_formatter(ScalarFormatter())
ax.xaxis.set_minor_locator(LogLocator(subs='auto'))
ax.yaxis.set_minor_locator(LogLocator(subs='auto'))
ax.tick_params(axis='both', which='major', labelsize=10,
length=5)
ax.tick_params(axis='both', which='minor', labelsize=0,
length=3)

# Statistical summary with improved formatting
n_comms = len(sizes)
s_med = int(np.median(sizes))
s_max = int(np.max(sizes))
s_mean = int(np.mean(sizes))

stats_text = (f"$n$ = {n_comms}\n"
              f"$\widetilde{{s}}$ = {s_med}\n"
              f"$s_{\rm max}$ = {s_max}")

ax.text(0.04, 0.05, stats_text,
        transform=ax.transAxes, fontsize=9,
        verticalalignment='bottom',
horizontalalignment='left',
        bbox=dict(boxstyle="round,pad=0.4",
                  facecolor='white', edgecolor='gray',
                  alpha=0.95, linewidth=0.8))

# Spine styling
for spine in ax.spines.values():
    spine.set_linewidth(1.2)

# Overall title (optional - often better in caption)
fig.suptitle("Community Size Distributions",
             fontsize=13, fontweight='bold', y=0.995)

# Add explanatory subtitle for statistics boxes and Y-axis
fig.text(0.5, 0.965,
        r'$P(S \geq s)$ is the fraction of communities with size'
        '$\geq s$. ' +
        r'Boxes show: $n$ = number of communities, $\widetilde{{s}}$

```

```

$ = median size, $s_{\rm max}$ = maximum size',
    ha='center', va='top', fontsize=9, style='italic',
color='#444444')

# Add panel labels (a, b, c, d)
panel_labels = ['(a)', '(b)', '(c)', '(d)']
for ax, label in zip(axes, panel_labels):
    ax.text(-0.15, 1.1, label, transform=ax.transAxes,
            fontsize=12, fontweight='bold', va='top')

# Create custom legend
from matplotlib.lines import Line2D
legend_elements = [
    Line2D([0], [0], color='#2E5EAA', linewidth=2.5,
label='Original graph'),
    Line2D([0], [0], color='#D4661F', linewidth=2.5,
label='Randomized graph'),
    Line2D([0], [0], color='black', linewidth=2.5, linestyle='--',
label='Louvain'),
    Line2D([0], [0], color='black', linewidth=2.5, linestyle='--',
label='SBM')
]
fig.legend(handles=legend_elements, loc='center',
           bbox_to_anchor=(0.5, -0.02), ncol=4, frameon=False,
           fontsize=10, columnspacing=1.5)

plt.tight_layout(rect=[0, 0.02, 1, 0.98])

# Save with high quality if path provided
if save_path:
    plt.savefig(save_path, dpi=600, bbox_inches='tight',
                facecolor='white', edgecolor='none')
    # Also save as PDF for vector graphics
    if not save_path.endswith('.pdf'):
        pdf_path = save_path.rsplit('.', 1)[0] + '.pdf'
        plt.savefig(pdf_path, bbox_inches='tight',
                    facecolor='white', edgecolor='none')

plt.show()

# Read partitions
with open('other_stuff/louvain_partition.txt', 'r') as f:
    partition = {int(line.split()[0]): int(line.split()[1]) for line
in f}
with open('other_stuff/graph_tool_partition.txt', 'r') as f:
    partition_gt = {int(line.split()[0]): int(line.split()[1]) for
line in f}
with open('other_stuff/louvain_partition_random.txt', 'r') as f:
    partition_louvain_random = {int(line.split()[0]): int(line.split())

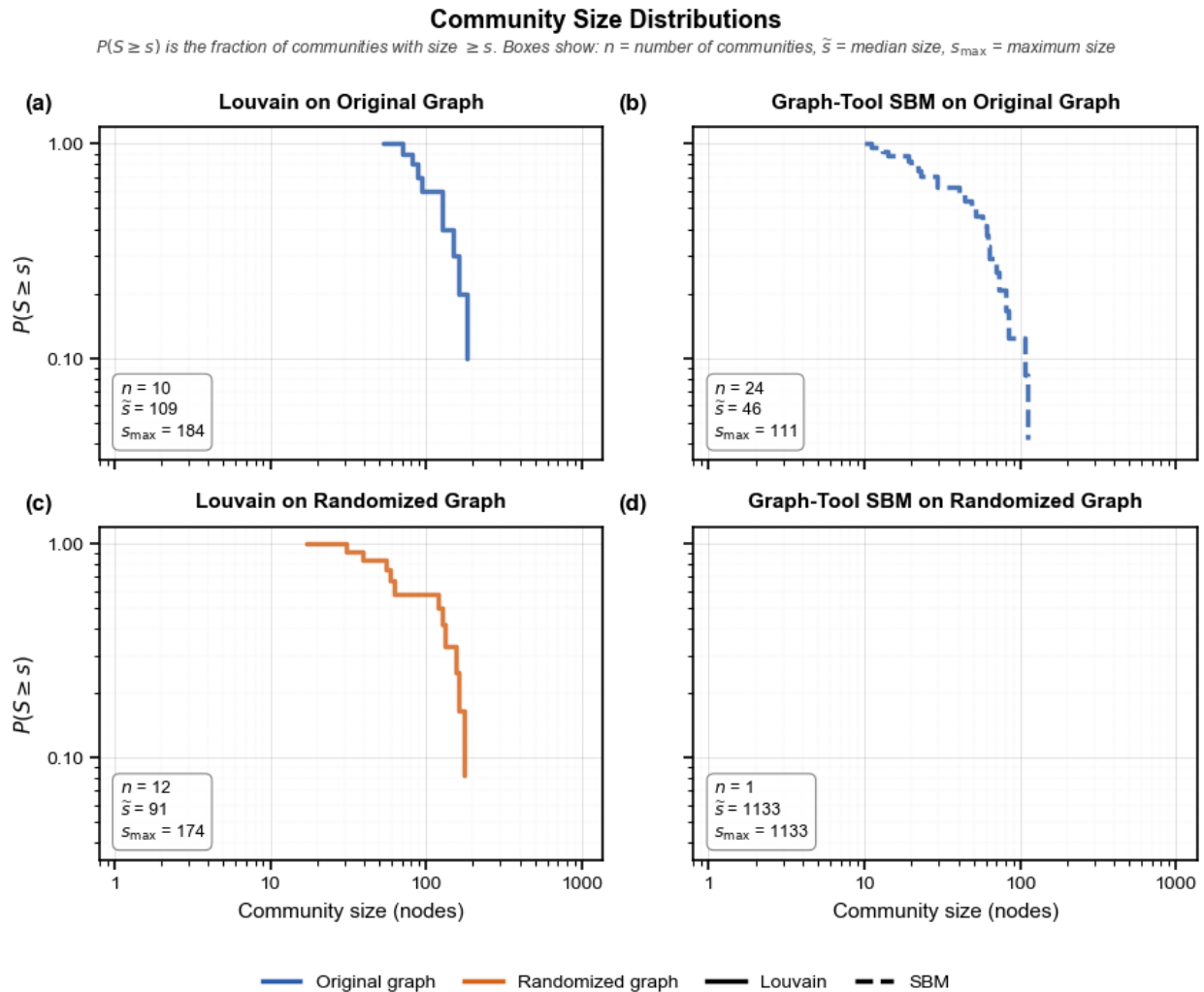
```



```
[1]) for line in f}
with open('other_stuff/graph_tool_partition_random.txt', 'r') as f:
    partition_gt_random = {int(line.split()[0]): int(line.split()[1])
    for line in f}

partitions = [partition, partition_gt, partition_louvain_random,
partition_gt_random]
titles = [
    "Louvain on Original Graph",
    "Graph-Tool SBM on Original Graph",
    "Louvain on Randomized Graph",
    "Graph-Tool SBM on Randomized Graph"
]

# Create plot and save
plot_community_size_distributions(partitions, titles)
```



the Y-axis represents the probability/fraction of communities that are at least as large as a given size, which is why all curves start at 1.0 (100% of communities have size ≥ 1) and decrease as size increases

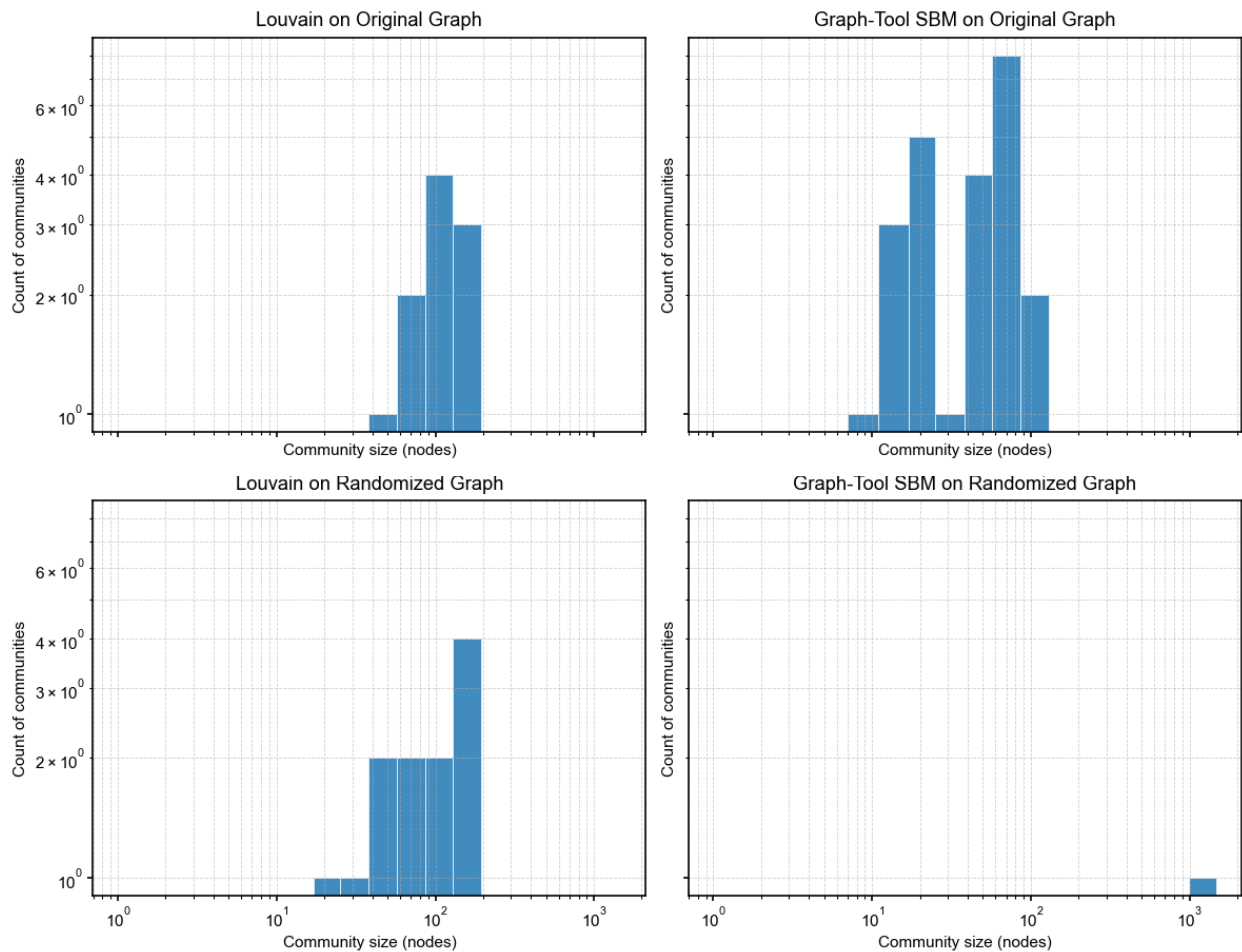
```
def _log_bins(max_size, base=1.5):
    # geometric progression of bin edges
    edges = [1]
    while edges[-1] < max_size:
        edges.append(edges[-1] * base)
    edges[-1] = max(edges[-1], max_size)
    return np.unique(np.floor(edges)).astype(int)

def plot_histograms(partitions, titles, base=1.5):
    all_sizes = [ _community_sizes(p) for p in partitions ]
    max_size = max(int(s.max()) for s in all_sizes)
    bins = _log_bins(max_size, base=base)

    fig, axes = plt.subplots(2, 2, figsize=(12, 10), sharex=True,
sharey=True)
    axes = axes.ravel()
    for ax, sizes, title in zip(axes, all_sizes, titles):
        ax.hist(sizes, bins=bins, alpha=0.85, edgecolor="white",
linewidth=0.6)
        ax.set_xscale("log")
        ax.set_yscale("log")
        ax.grid(True, which="both", linestyle="--", linewidth=0.6,
alpha=0.6)
        ax.set_xlabel("Community size (nodes)")
        ax.set_ylabel("Count of communities")
        ax.set_title(title, pad=8)
    fig.suptitle("Community Size Distributions (log-binned
histogram)", y=0.98, fontsize=14)
    fig.tight_layout(rect=[0, 0, 1, 0.96])
    plt.show()

plot_histograms(partitions, titles, base=1.5)
```

Community Size Distributions (log-binned histogram)



Another figure showing the histogram of community sizes for each method would also be informative, as it would illustrate the absolute counts of communities of different sizes rather than their relative proportions. I made this fig very simpler.

e)

The most widely used metrics for comparing graph partitions include Normalized Mutual Information (NMI), Adjusted Mutual Information (AMI), Adjusted Rand Index (ARI), and Variation of Information (VI).

Based on established methods, an ideal partition comparison function should have these properties:

1. Symmetry The function should be symmetric: $f(b_1, b_2) = f(b_2, b_1)$, meaning the order of comparison doesn't matter.
2. Normalization and Interpretability Metrics like NMI are normalized to return values between 0 and 1, where 0 indicates no agreement and 1 indicates perfect agreement. This makes results comparable across different network sizes and numbers of communities.

3. **Adjustment for Chance** The baseline value between random clusterings should be corrected for chance agreement, as metrics like AMI and ARI do by computing expected values under random partitioning. Without adjustment, measures like NMI show monotonically increasing patterns as the number of clusters increases, even for random partitions.
4. **Metric Properties (for distance measures)** For distance-based measures like Variation of Information, the function should be a true metric satisfying the triangle inequality, which means VI is positive, symmetric, and obeys $f(b_1, b_3) \leq f(b_1, b_2) + f(b_2, b_3)$ Comparing clusterings—an information based distance.
5. **Label Independence** The metric should be independent of label permutations—relabeling communities shouldn't change the similarity score.

We will explain Adjusted Mutual Information (AMI) here. Because:

- AMI corrects for chance agreement by computing Expected Mutual Information under a hypergeometric model of randomness.
- AMI is particularly effective when there are pure, well-defined clusters in the solution.
- It's bounded $[0, 1]$ for easy interpretation.
- Available in standard libraries (scikit-learn)

```
from sklearn.metrics.cluster import adjusted_mutual_info_score

def compare_partitions(partition1, partition2):
    """
    Compare two partitions using Adjusted Mutual Information (AMI).
    AMI is a measure from information theory that quantifies the
    similarity between two partitions,
    adjusted for chance. It ranges from 0 (no mutual information) to 1
    (perfect correlation).
    """

    # Extract community labels for each node
    nodes =
    set(partition1.keys()).intersection(set(partition2.keys()))
    labels1 = [partition1[n] for n in nodes]
    labels2 = [partition2[n] for n in nodes]

    # Compute AMI
    ami = adjusted_mutual_info_score(labels1, labels2)
    return ami

# Example usage:
ami_louvain_gt = compare_partitions(partition, partition_gt)
ami_louvain_random = compare_partitions(partition,
partition_louvain_random)
ami_gt_random = compare_partitions(partition_gt, partition_gt_random)
ami_louvain_gt_random = compare_partitions(partition_louvain_random,
partition_gt_random)
print(f"AMI between Louvain and Graph-Tool SBM on original graph:
{ami_louvain_gt:.4f}")
```

```
print(f"AMI between Louvain on original and randomized graph:
{ami_louvain_random:.4f}")
print(f"AMI between Graph-Tool SBM on original and randomized graph:
{ami_gt_random:.4f}")
print(f"AMI between Louvain and Graph-Tool SBM on randomized graph:
{ami_louvain_gt_random:.4f}")
```

```
AMI between Louvain and Graph-Tool SBM on original graph: 0.6045
AMI between Louvain on original and randomized graph: 0.0010
AMI between Graph-Tool SBM on original and randomized graph: 0.0000
AMI between Louvain and Graph-Tool SBM on randomized graph: 0.0000
```

We can see that the AMI between Louvain and Graph-tool is near 0.61, which indicates a moderate level of agreement between the two clustering results. This suggests that while there are some similarities in the clusters identified by both algorithms, there are also significant differences. The NMI value reflects that the partitions are not identical but share some common structure.

Question 3

We want to study Lancichinetti-Fortunato-Radicchi (LFR) benchmark for community detection in networks. The LFR benchmark generates synthetic networks with predefined community structures, allowing researchers to evaluate the performance of community detection algorithms.

a)

the paper motivation is that, well first, community detection is difficult in large networks, so it's important to have benchmarks to test algorithms. The LFR benchmark is designed to create synthetic networks that mimic real-world properties, like power-law degree distributions and community size distributions. This helps researchers evaluate how well different algorithms can identify communities in complex networks. They account for this heterogeneity by allowing for varying community sizes and node degrees in the generated networks.

Their main contribution is that they developed a benchmark that generates networks with realistic properties, including power-law distributions for both node degrees and community sizes. This allows for more accurate testing of community detection algorithms. They also introduced a mixing parameter (μ) to control the strength of community structure, which helps in evaluating algorithm performance under different conditions.

The inputs to the LFR benchmark include:

- **Number of Nodes (N):** The total number of nodes in the network.
- **Average Degree (k):** The average number of connections (edges) each node has.
- **Maximum Degree (k_max):** The maximum number of connections a node can have.
- **Mixing Parameter (μ):** This parameter controls the ratio of inter-community edges to intra-community edges. A lower value of μ indicates stronger community structure, while a higher value indicates weaker community structure.

- **Community Size Distribution:** The sizes of the communities can follow a power-law distribution, allowing for the creation of communities of varying sizes.
- **Degree Distribution:** The degree distribution of the nodes can also follow a power-law distribution, which is common in real-world networks. By adjusting these parameters, researchers can create networks that mimic the properties of real-world networks, such as social networks, biological networks, and information networks. The LFR benchmark is widely used to test and compare the effectiveness of different community detection algorithms under various conditions.

The output is a synthetic network represented as a graph, where nodes are connected by edges, and the community structure is defined based on the specified parameters. The generated network can then be used to test and evaluate community detection algorithms.

b)

```
def generate_powerlaw_degree_sequence(n, gamma, avg_k):
    """
    Generate a power-law degree sequence for a network of size n,
    with degree exponent gamma and average degree avg_k.
    """
    # Calculate the minimum degree k_min using the average degree
    formula
    if gamma <= 2:
        raise ValueError("Gamma must be greater than 2 for a valid
        power-law distribution.")

    k_min = (gamma - 2) * avg_k / (gamma - 1)

    # Generate the degree sequence
    degrees = []
    while len(degrees) < n:
        # Sample from the power-law distribution using inverse
        transform sampling
        r = np.random.uniform(0, 1)
        k = int(k_min * (1 - r) ** (-1 / (gamma - 1)))
        if k >= 1: # Ensure degree is at least 1
            degrees.append(k)

    # Adjust the degree sequence to ensure the sum of degrees is even
    if sum(degrees) % 2 != 0:
        degrees[0] += 1 # Increment the first degree to make the sum
    even

    # Ensure sum of degrees is approximately n * avg_k
    # current_avg_k = sum(degrees) / n
    # scaling_factor = avg_k / current_avg_k
    # degrees = [max(1, int(k * scaling_factor)) for k in degrees]

    return degrees
```

```

# Example usage:
n = 1000          # network size
gamma = 2.5       # powerlaw degree exponent
avg_k = 10        # average degree
degree_sequence = generate_powerlaw_degree_sequence(n, gamma, avg_k)
print(f"Generated degree sequence of length {len(degree_sequence)}
with average degree {np.mean(degree_sequence):.2f}")

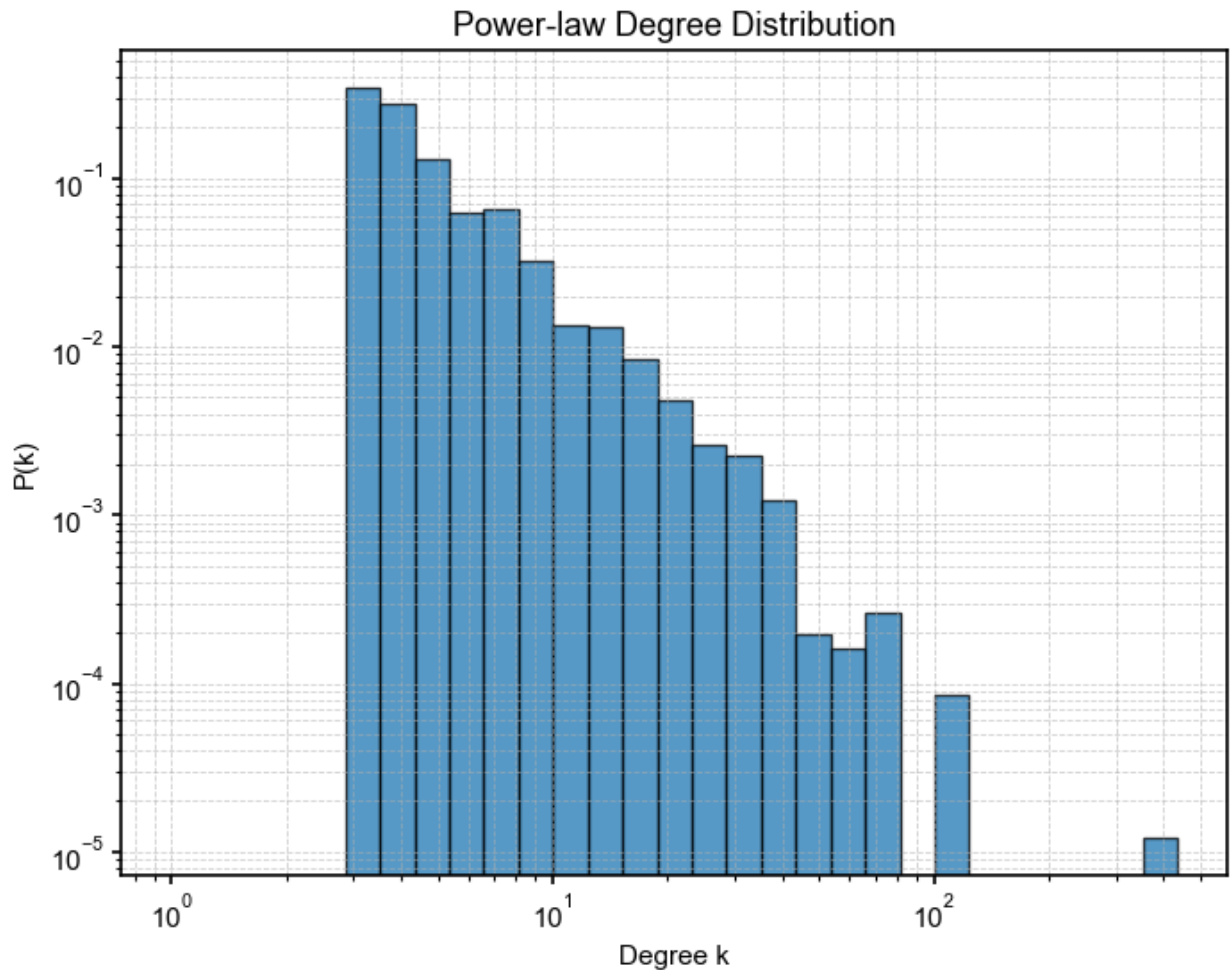
```

Generated degree sequence of length 1000 with average degree 8.30

```

# Plot the degree distribution
plt.figure(figsize=(8, 6))
plt.hist(degree_sequence, bins=np.logspace(np.log10(1),
np.log10(max(degree_sequence)), 30), density=True, alpha=0.75,
edgecolor='black')
plt.xscale('log')
plt.yscale('log')
plt.xlabel('Degree k')
plt.ylabel('P(k)')
plt.title('Power-law Degree Distribution')
plt.grid(True, which="both", linestyle="--", linewidth=0.6, alpha=0.6)
plt.show()

```



```

from typing import Sequence
import math
from typing import List, Tuple, Optional

def split_internal_external(
    degrees: Sequence[int],
    mu: float,
    seed: Optional[int] = None
) -> Tuple[List[int], List[int]]:
    """
        Given degrees  $k_i$  and mixing parameter  $\mu$ , return integer  $(k_{in_i}, k_{out_i})$ 
        so that  $k_{in_i} + k_{out_i} = k_i$  and  $E[k_{out_i}] = \mu * k_i$ .

        Uses unbiased stochastic rounding to minimize aggregate mismatch.

        Parameters
        -----
        degrees : Sequence[int]
            Degree sequence.
    
```



```

mu : float
    Mixing parameter in [0,1]. Fraction of stubs earmarked for
    'external' edges.
seed : int, optional
    Random seed.

Returns
-----
(kin, kout) : (List[int], List[int])
    Integer internal and external stub counts per node.
"""
if seed is not None:
    random.seed(seed)
kin, kout = [], []
for k in degrees:
    # desired internal: k*(1-mu), external: k*mu
    desired_in = (1.0 - mu) * k
    fin = math.floor(desired_in)
    pin = desired_in - fin
    k_in = fin + (1 if random.random() < pin else 0)
    k_in = max(0, min(k_in, k))
    k_out = k - k_in
    kin.append(k_in)
    kout.append(k_out)

return kin, kout

# test
n = 1000
gamma = 2.5
avg_k = 10
mu = 0.3 # Mixing parameter
degree_sequence = generate_powerlaw_degree_sequence(n, gamma, avg_k)
kin, kout = split_internal_external(degree_sequence, mu, seed=42)
for i in range(10):
    print(f"Node {i}: Degree={degree_sequence[i]}, Internal
Edges={kin[i]}, External Edges={kout[i]}")

```

```

Node 0: Degree=9, Internal Edges=6, External Edges=3
Node 1: Degree=4, Internal Edges=3, External Edges=1
Node 2: Degree=3, Internal Edges=2, External Edges=1
Node 3: Degree=5, Internal Edges=4, External Edges=1
Node 4: Degree=3, Internal Edges=2, External Edges=1
Node 5: Degree=4, Internal Edges=3, External Edges=1
Node 6: Degree=4, Internal Edges=2, External Edges=2
Node 7: Degree=4, Internal Edges=3, External Edges=1
Node 8: Degree=4, Internal Edges=3, External Edges=1
Node 9: Degree=8, Internal Edges=6, External Edges=2

```

```

def generate_powerlaw_community_sizes(num_communities, gamma,
    avg_size):
    """
    Generate a power-law community size distribution.

    Parameters:
    - num_communities: Number of communities.
    - gamma: Power-law exponent.
    - avg_size: Average community size.

    Returns:
    - community_sizes: List of community sizes.
    """
    # Calculate the minimum community size
    if gamma <= 2:
        raise ValueError("Gamma must be greater than 2 for a valid
power-law distribution.")

    size_min = (gamma - 2) * avg_size / (gamma - 1)

    community_sizes = []
    while len(community_sizes) < num_communities:
        r = np.random.uniform(0, 1)
        size = int(size_min * (1 - r) ** (-1 / (gamma - 1)))
        if size >= 1:
            community_sizes.append(size)

    return community_sizes

num_communities = 10
gamma = 2.5
avg_size = 50
community_sizes = generate_powerlaw_community_sizes(num_communities,
gamma, avg_size)
print(f"Generated community sizes: {community_sizes}")

Generated community sizes: [128, 17, 34, 48, 32, 20, 25, 55, 22, 20]

```

Two extra constraints are required when choosing the min/max community sizes:

Choose the minimum community size $s_{\min}$ and $s_{\max}$ so that

$s_{\min} \geq k_{\min}$ and $s_{\max} \geq k_{\max}$.

Reminder: $s_{\min}$ and $s_{\max}$ are the minimum and maximum community sizes, while $k_{\min}$ and $k_{\max}$ are the minimum and maximum node degrees.

This guarantees “that a node of any degree can be included in at least a community,” i.e., every node’s internal degree requirement can be met within some community size. The paper states this explicitly when describing Step 3 (and also reminds that the total of all community sizes must equal N)

c)

```
# Use nx.LFR benchmark graph() to generate a synthetic graph with
known community structure
#
# # Parameters for LFR benchmark
n = n                # number of nodes
tau1 = gamma         # power-law exponent for degree distribution
tau2 = 1.5           # power-law exponent for community size
distribution
mu = 0.1             # mixing parameter
average_degree = avg_k # average degree
max_degree = 50      # maximum degree
min_community = 20    # minimum community size
max_community = 100   # maximum community size
# Generate LFR benchmark graph
G_lfr = nx.LFR_benchmark_graph(n, tau1, tau2, mu,
                                average_degree=average_degree,
                                max_degree=max_degree,
                                min_community=min_community,
                                max_community=max_community,
                                seed=42)

# Extract ground truth communities
ground_truth_partition = {}
for node in G_lfr.nodes():
    ground_truth_partition[node] = list(G_lfr.nodes[node]
    ['community'])[0] # take one community if overlapping
# Perform community detection using Louvain method
detected_partition = community_louvain.best_partition(G_lfr,
    random_state=42)
# Compare detected communities with ground truth using NMI
nmi_lfr = compare_partitions(ground_truth_partition,
    detected_partition)
print(f"NMI between detected communities and ground truth in LFR
benchmark: {nmi_lfr:.4f}")

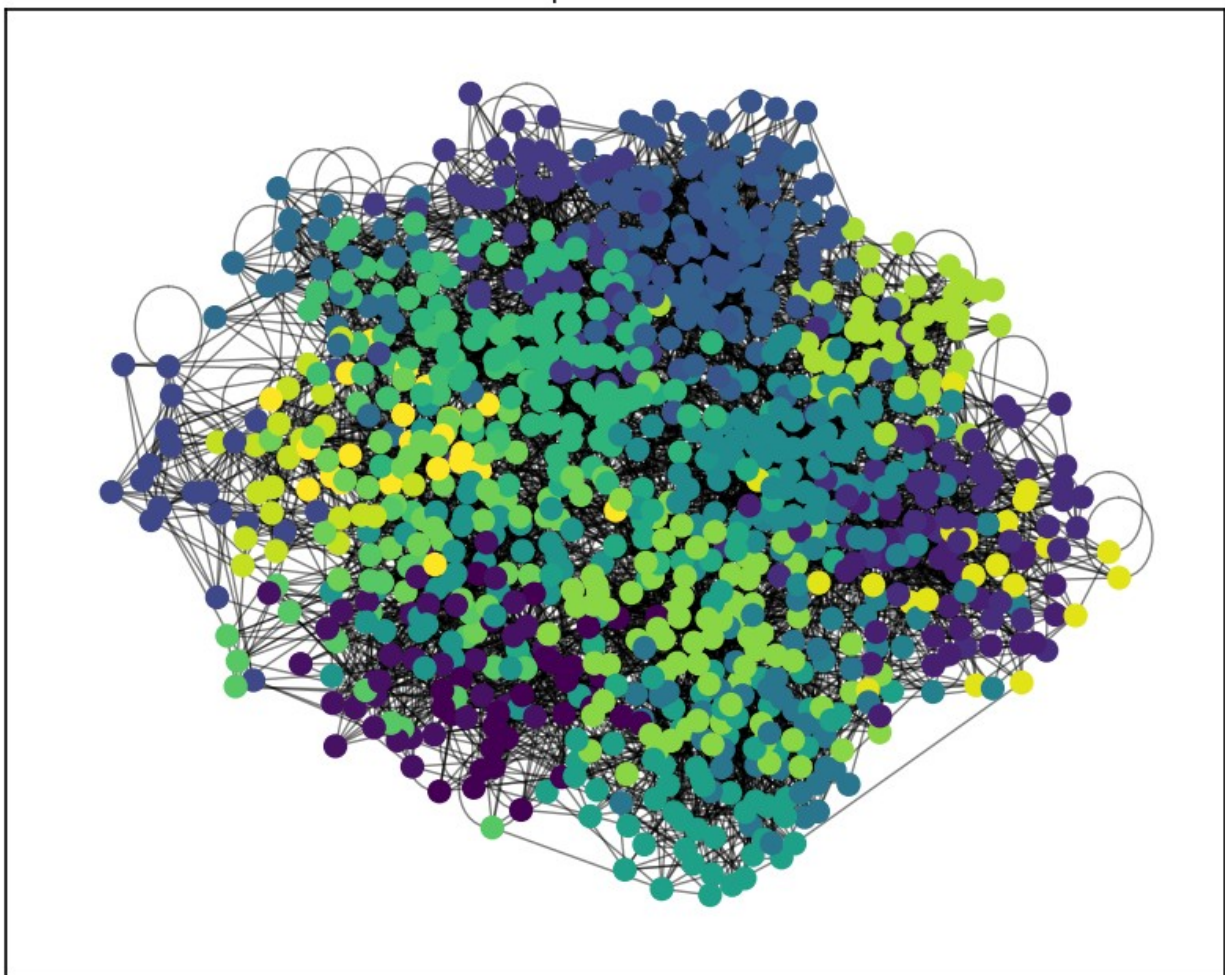
NMI between detected communities and ground truth in LFR benchmark:
1.0000
```

Networkx LFR function convergence most often struggles when your parameters make the integer constraints impossible or barely feasible: 1) Degree vs. community-size feasibility. A necessary condition is that $s_{\min} - 1 \geq k_{\max} * (1 - \mu)$. If you violate this (e.g., large hubs + small communities + small μ), the generator will burn rewiring budget and often quit without success.)

2) Mixing parameter (μ) too small or too large. Extremely low or high μ values can make it hard to satisfy internal/external degree constraints, leading to non-convergence.

```
# Visualize the LFR benchmark graph with detected communities
plt.figure(figsize=(10, 8))
pos = nx.spring_layout(G_lfr)
num_communities_lfr = len(set(detected_partition.values()))
cmap = plt.get_cmap("viridis", num_communities_lfr)
nx.draw_networkx_nodes(G_lfr, pos, detected_partition.keys(),
node_size=100, cmap=cmap,
node_color=list(detected_partition.values()))
nx.draw_networkx_edges(G_lfr, pos, alpha=0.5)
plt.title("LFR Benchmark Graph with Detected Communities")
plt.show()
```

LFR Benchmark Graph with Detected Communities



```
# Get degree sequence of the LFR graph
degree_sequence_lfr = [d for n, d in G_lfr.degree()]
print(f"LFR graph degree sequence length: {len(degree_sequence_lfr)}
with average degree {np.mean(degree_sequence_lfr):.2f}")

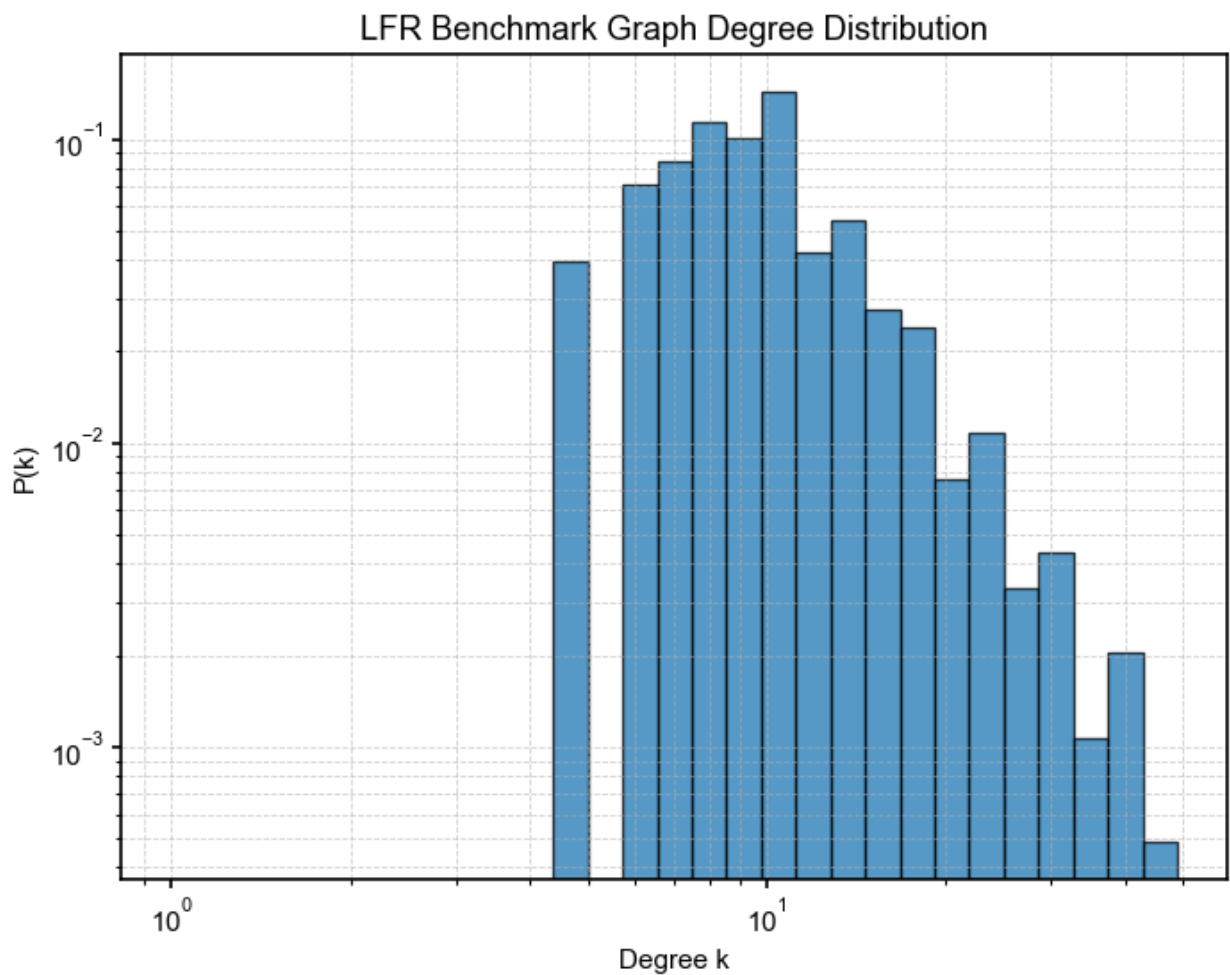
# plot the Power-law Degree Distribution of the LFR graph
```

```

plt.figure(figsize=(8, 6))
plt.hist(degree_sequence_lfr, bins=np.logspace(np.log10(1),
np.log10(max(degree_sequence_lfr)), 30), density=True, alpha=0.75,
edgecolor='black')
plt.xscale('log')
plt.yscale('log')
plt.xlabel('Degree k')
plt.ylabel('P(k)')
plt.title('LFR Benchmark Graph Degree Distribution')
plt.grid(True, which="both", linestyle="--", linewidth=0.6, alpha=0.6)
plt.show()

```

LFR graph degree sequence length: 1000 with average degree 12.19



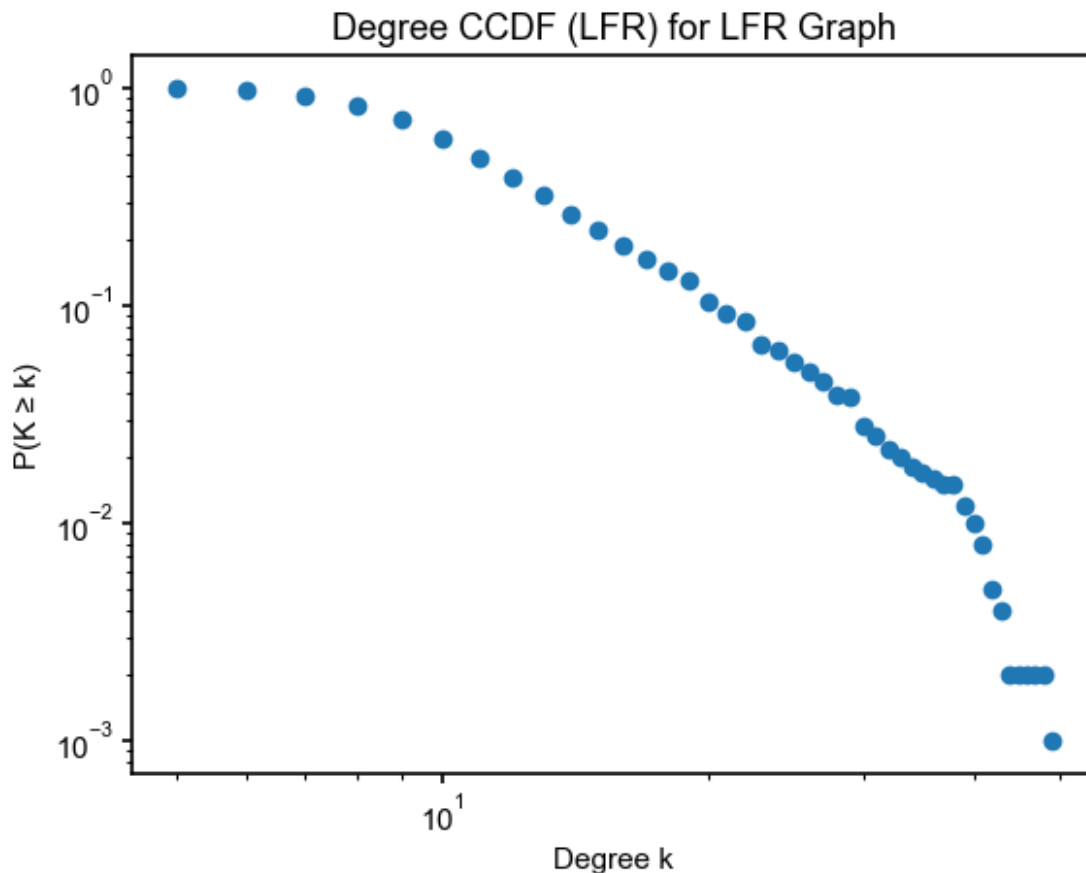
```

# CCDF
deg = np.array([d for _, d in nx.degree(G_lfr)], dtype=int)

ks = np.arange(deg.min(), deg.max()+1)
ccdf = np.array([(deg >= k).mean() for k in ks])

```

```
plt.figure()
plt.loglog(ks, ccdf, marker='o', linestyle='none')
plt.xlabel('Degree k'); plt.ylabel('P(K ≥ k)')
plt.title('Degree CCDF (LFR) for LFR Graph')
plt.show()
```



Question 4

I created a new repository for this new library: <https://github.com/sepehrilami/netsci-toolkit>

I added a comprehensive README file that includes installation instructions, usage examples, and documentation.

I revised my previous functions and created some extra functions to enhance the library's functionality. I ensured that the code is well-documented with comments and docstrings for better understanding.

It was actually my first time creating a package, so I had to learn the process, and now I really love to improve the package and add more functionalities to it, and hopefully, release it to PyPI in the future and make it available for everyone to use easily! :D

a-d)

```
# Let's try the new toolkit:
!pip install git+https://github.com/sepehrilami/netsci-toolkit.git

Collecting git+https://github.com/sepehrilami/netsci-toolkit.git
  Cloning https://github.com/sepehrilami/netsci-toolkit.git to
/private/var/folders/nw/_gc7dkbx2gx_j7xr0bxs1x4h0000gn/T/pip-req-
build-ec7nm3g4
  Running command git clone --filter=blob:none --quiet
https://github.com/sepehrilami/netsci-toolkit.git
/private/var/folders/nw/_gc7dkbx2gx_j7xr0bxs1x4h0000gn/T/pip-req-
build-ec7nm3g4
  Resolved https://github.com/sepehrilami/netsci-toolkit.git to commit
0171c2b2805e3c3deb0d4a18a46c6f863b9f833c
  Installing build dependencies ... ents to build wheel ... etadata
(pyproject.toml) ... ent already satisfied: networkx>=2.5 in
/Users/sepehr/miniconda/envs/gt/lib/python3.12/site-packages (from
netsci-toolkit==0.1.0) (3.5)

import netsci_toolkit as nst

# Load or create a graph
G = nx.karate_club_graph()

# Detect communities using Louvain method
result = nst.detect_communities_louvain(G)
print(f"Found {result['num_communities']} communities")
print(f"Modularity: {result['modularity']:.4f}")

# Detect communities using other methods
result_gn = nst.detect_communities_girvan_newman(G, num_communities=2)
result_lp = nst.detect_communities_label_propagation(G)
result_gm = nst.detect_communities_greedy_modularity(G)

# Analyze community structure
analysis = nst.analyze_communities(G, result['communities'])
print(f"Average community size: {analysis['avg_community_size']:.2f}")
print(f"Community sizes: {analysis['community_sizes']}")

# Compare different methods
comparison = nst.compare_communities(G, method="all")
for method, res in comparison.items():
    print(f"{method}: {res['num_communities']} communities,
modularity={res['modularity']:.4f}")

# Get node-to-community mapping
mapping = nst.get_node_community_mapping(result['communities'])
print(f"Node 0 is in community: {mapping[0]}")
```



```
Found 4 communities
Modularity: 0.4449
Average community size: 8.50
Community sizes: [11, 5, 6, 12]
louvain: 4 communities, modularity=0.4439
girvan_newman: 2 communities, modularity=0.3477
label_propagation: 3 communities, modularity=0.3095
greedy_modularity: 3 communities, modularity=0.4110
Node 0 is in community: 0
```

Now let's convert a networkx graph to a graph-tool graph. There are two functions created for this purpose in the package: `nx_to_gt` and `convert_nx_to_graph_tool`.

`nx_to_gt` gets a networkx graph as input and returns a tuple with two outputs: 1) `g`: `graph_tool.Graph` - The converted graph-tool graph 2) `nx2gt`: dict - Dictionary mapping NetworkX node IDs to graph-tool vertices

`convert_nx_to_graph_tool` is a more user-friendly function that directly returns the converted graph-tool graph without the mapping dictionary. It takes a NetworkX graph as input and returns the corresponding graph-tool graph.

Let's try them both:

```
import graph_tool.all as gt
import netsci_toolkit as nst
import networkx as nx

# use nst to convert networkx graph to graph-tool
g_gt, nx2gt = nst.nx_to_gt(G)
print(type(g_gt))
print(type(nx2gt))

g_gt2 = nst.convert_nx_to_graph_tool(G)
print(type(g_gt2))

# check if g_gt and g_gt2 are the same
print(f'both graphs return the same graph-tool graph:
{g_gt.num_edges() == g_gt2.num_edges()}')

# Do community detection using graph-tool
# Detect communities using Stochastic Block Model
result = nst.detect_communities_graph_tool(G,
method="stochastic_block_model")
print(f"Found {result['num_communities']} communities")

# Convert NetworkX graph to graph-tool format
gt_graph = nst.convert_nx_to_graph_tool(G)

<class 'graph_tool.Graph'>
<class 'dict'>
```



```
<class 'graph_tool.Graph'>  
both graphs return the same graph-tool graph: True  
Found 1 communities
```

Question 5

Thank you again for asking for our feedback. I think this assignment was VERY time-consuming. I had to spend more than two full days on it, and it was too much since we all have many other research projects and assignments. I think just questions 2 and 4 were really enough for this assignment.

The lectures are also good, but I think going more in depth in some topics would be helpful (but I know that's now the goal of this course, so, I think lectures are actually useful and should remain as is).